

# Equations, Implementation, and References for `rlv_sim` 6-DOF Simulation

## 1 Rigid-Body Dynamics

### 1.1 Euler's Rotational Equation of Motion

Code: `dynamics.py` – `compute_angular_acceleration`

```
Iomega = I_tensor @ omega
gyroscopic = np.cross(omega, Iomega)
return I_inv @ (torque - gyroscopic)
```

**Equation:**

$$I\dot{\omega} = \tau - \omega \times (I\omega) \implies \dot{\omega} = I^{-1}(\tau - \omega \times (I\omega))$$

**Explanation:** This is the standard Euler equation for a rigid body about its principal axes. The cross-product order ( $\omega \times I\omega$ ) and sign are correct. The inertia tensor  $I$  is recomputed each integration sub-step as a function of instantaneous propellant mass, which is a defensible variable-mass simplification because the omitted  $-\dot{I}\omega$  term is negligible relative to the attitude control dynamics (propellant changes over tens of seconds, while attitude dynamics act on sub-second time scales).

**References:** Goldstein, *Classical Mechanics*; Zipfel [6, Ch. 4]; Wie [5, Ch. 3].

### 1.2 Newton's Second Law (Translational)

Code: `dynamics.py` – `compute_linear_acceleration`

```
force_breakdown = _compute_force_breakdown(r, v, q, m, ...)
return force_breakdown['total'] / m
```

**Equation:**

$$\ddot{\mathbf{r}} = \frac{\mathbf{F}_{\text{grav}} + \mathbf{F}_{\text{thrust}} + \mathbf{F}_{\text{drag}} + \mathbf{F}_{\text{lift}} + \mathbf{F}_{\text{grid fin}}}{m}$$

**Explanation:** The state is propagated in the Earth-Centered Inertial (ECI) frame, which is inertial; therefore no Coriolis or centrifugal pseudo-forces appear. Earth's rotation correctly enters only through the air-relative velocity used in aerodynamic force computations (see Section 4.8). This matches the standard Newtonian formulation in an inertial frame.

**References:** Vallado [1, Ch. 3]; Curtis [2, Ch. 2].

## 2 Attitude Kinematics (Quaternions)

Convention:  $\mathbf{q} = [w, x, y, z]$ , scalar-first, Hamilton product, body-to-inertial.

### 2.1 Quaternion Kinematic Differential Equation

Code: frames.py – omega\_matrix and quaternion\_derivative

```
def omega_matrix(omega):
    wx, wy, wz = omega
    return np.array([[0.0, -wx, -wy, -wz],
                    [wx, 0.0, wz, -wy],
                    [wy, -wz, 0.0, wx],
                    [wz, wy, -wx, 0.0]])

def quaternion_derivative(q, omega):
    Omega = omega_matrix(omega)
    return 0.5 * Omega @ q
```

**Equation:**

$$\dot{\mathbf{q}} = \frac{1}{2} \Omega(\omega) \mathbf{q}$$

**Explanation:** This is the standard Hamilton-convention quaternion kinematic equation. The matrix  $\Omega(\omega)$  is constructed so that  $\Omega(\omega)q$  is identical to the Hamilton product  $q \otimes [0, \omega]$ , ensuring internal consistency with the multiplication convention used throughout the code.

**References:** Markley & Crassidis [3, Ch. 2]; Wertz [4, Ch. 7].

### 2.2 Quaternion (Hamilton) Product

Code: frames.py – quaternion\_multiply

```
return np.array([
    w1*w2 - x1*x2 - y1*y2 - z1*z2,
    w1*x2 + x1*w2 + y1*z2 - z1*y2,
    w1*y2 - x1*z2 + y1*w2 + z1*x2,
    w1*z2 + x1*y2 - y1*x2 + z1*w2
])
```

**Equation (vector form):**

$$q_1 \otimes q_2 = \begin{bmatrix} w_1 w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2 \\ w_1 \mathbf{v}_2 + w_2 \mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2 \end{bmatrix}$$

**Explanation:** Expanding the vector form component-by-component reproduces the code exactly, confirming the correct Hamilton product convention.

**References:** Markley & Crassidis [3, Ch. 2]; Vallado [1, Ch. 3].

## 2.3 Quaternion → Rotation Matrix

Code: `frames.py` – `quaternion_to_rotation_matrix`

```
return np.array([
    [1 - 2*(yy + zz), 2*(xy - wz), 2*(xz + wy)],
    [2*(xy + wz), 1 - 2*(xx + zz), 2*(yz - wx)],
    [2*(xz - wy), 2*(yz + wx), 1 - 2*(xx + yy)]
])
```

**Equation:**

$$R = \begin{bmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{bmatrix}$$

**Explanation:** This is the standard body-to-inertial direction cosine matrix derived from a Hamilton quaternion. All off-diagonal signs are correct.

**References:** Wikipedia “Quaternions and spatial rotation”; Markley & Crassidis [3, Ch. 2]; Vallado [1, Ch. 3].

## 2.4 Rotation Matrix → Quaternion (Shepperd’s Method)

Code: `frames.py` – `rotation_matrix_to_quaternion` (The implementation branches on the trace and diagonal dominance to avoid division by near-zero quantities.)

**Equation:** Four-case branch algorithm (trace  $> 0$ , or largest diagonal element  $R_{00}, R_{11}, R_{22}$ ). Each case computes  $s = \frac{1}{2}\sqrt{1 + \text{tr}(R)}$ , etc.

**Explanation:** This is Shepperd’s numerically stable algorithm for extracting a quaternion from a rotation matrix. It avoids catastrophic cancellation by selecting the case with the largest denominator.

**References:** Shepperd [11]; Vallado [1, Ch. 3]; Markley & Crassidis [3, Ch. 2].

## 2.5 Quaternion Error (Attitude Feedback)

Code: `frames.py` – `quaternion_error`

```
q_inv = quaternion_inverse(q_current)
q_err = quaternion_multiply(q_inv, q_desired)
if q_err[0] < 0:
    q_err = -q_err
```

**Equation:**

$$\mathbf{q}_{\text{err}} = \mathbf{q}_{\text{current}}^{-1} \otimes \mathbf{q}_{\text{desired}}, \quad \text{with sign chosen so that } w_{\text{err}} \geq 0 \text{ (shortest path)}$$

**Explanation:** This definition yields the body-frame attitude error used in quaternion feedback control. The sign-flip enforces the shortest-path rotation, preventing “unwinding” (commanding the long way around).

**References:** Markley & Crassidis [3, Ch. 6]; Wie [5, Ch. 5].

## 3 Numerical Integration

### 3.1 Classical Fourth-Order Runge-Kutta (RK4)

Code: `integrators.py – rk4_step`

```
k1 = state_derivative_vector(y, t, torque, ctx)
y2 = y + 0.5*dt*k1; k2 = state_derivative_vector(y2, t+0.5*dt, torque, ctx)
y3 = y + 0.5*dt*k2; k3 = state_derivative_vector(y3, t+0.5*dt, torque, ctx)
y4 = y + dt*k3; k4 = state_derivative_vector(y4, t+dt, torque, ctx)
y_new = y + (dt/6.0)*(k1 + 2*k2 + 2*k3 + k4)
```

Equations (Butcher tableau):

$$\begin{aligned}k_1 &= f(t, y), \\k_2 &= f\left(t + \frac{h}{2}, y + \frac{h}{2}k_1\right), \\k_3 &= f\left(t + \frac{h}{2}, y + \frac{h}{2}k_2\right), \\k_4 &= f(t + h, y + hk_3), \\y_{n+1} &= y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).\end{aligned}$$

**Explanation:** The nodes and weights match the classical RK4 scheme. The quaternion component is re-normalised after every sub-stage, a standard practice to prevent numerical drift from the unit-norm constraint.

**References:** Press et al. [12, Ch. 17]; Butcher, *Numerical Methods for Ordinary Differential Equations*.

## 4 Environment Model

### 4.1 U.S. Standard Atmosphere 1976

Code: `forces.py – _build_us76_tables` and `compute_atmosphere_properties`

```
US76_H = [0, 11000, 20000, 32000, 47000, 51000, 71000, 84852] # m
US76_L = [-0.0065, 0.0, 0.0010, 0.0028, 0.0, -0.0028, -0.0020] # K/m
if abs(lapse) > 1e-12:
    T = T0 + lapse * dh
    P = P0 * (T / T0) ** (-G0 / (lapse * R_gas))
else:
    T = T0
    P = P0 * exp(-G0 * dh / (R_gas * T0))
```

**Equations:** For each layer with constant lapse rate  $L$ :

$$T = T_0 + L h, \quad P = P_0 \left( \frac{T}{T_0} \right)^{-\frac{g_0}{LR}}$$

For isothermal layers ( $L = 0$ ):

$$T = T_0, \quad P = P_0 \exp\left(-\frac{g_0 h}{R T_0}\right)$$

**Explanation:** The US76 model is implemented exactly according to its definition, using piecewise linear temperature gradients. Density is derived from the ideal gas law  $\rho = P/(RT)$ .

**References:** NOAA, NASA, USAF [9]; Vallado [1, App. B].

## 4.2 Central Gravity and $J_2$ Perturbation

**Code:** forces.py – compute\_gravity\_acceleration

```
r_mag = np.linalg.norm(r)
r_hat = r / r_mag
# Central term
a_central = -C.MU_EARTH / r_mag**2 * r_hat
# J2 term (corrected to use equatorial radius, include mu, and correct r powers)
j2_factor = -1.5 * C.J2 * C.MU_EARTH * (C.R_EARTH_EQUATORIAL**2) / (r_mag**4)
a_j2 = j2_factor * ( (5*(r_hat[2])**2 - 1) * r_hat - 2*(r_hat[2]) * np.array([0,0,1]) )
return a_central + a_j2
```

**Equations (central +  $J_2$ ):**

$$\mathbf{a}_{\text{central}} = -\frac{\mu}{r^3}\mathbf{r}$$

$$\mathbf{a}_{J_2} = -\frac{3}{2}J_2\frac{\mu R_{\oplus}^2}{r^4}\left[\left(5\frac{z^2}{r^2} - 1\right)\frac{\mathbf{r}}{r} - 2\frac{z}{r}\hat{\mathbf{z}}\right]$$

**Explanation:** The central term uses the standard gravitational parameter  $\mu$ . The  $J_2$  term is the classical zonal oblateness perturbation, and crucially, it is normalised using the **equatorial** radius  $R_{\oplus}$  (WGS84 value) – a correction made during this verification process. Previously a mean radius was used, underestimating the perturbation by 0.22%.

**References:** Vallado [1, Ch. 8]; Curtis [2, Eq. 12.30]; Lemoine et al. [10].

## 4.3 EGM96 Spherical-Harmonic Gravity (Defect Found and Fixed)

**Code:** forces.py – compute\_egm96\_gravity\_accel

Three independent defects were identified and corrected:

1. **Coefficient scale error:**  $\bar{C}_{2,0}$  was stored as ‘-0.484165143403758e+00’ instead of ‘-0.484165143403758e-03’. Corrected to the proper value  $-J_2/\sqrt{5} \approx -4.8417 \times 10^{-4}$ .
2. **Angular acceleration prefactor:** The  $\phi$  and  $\lambda$  acceleration components were multiplied by  $\mu/r$  instead of  $\mu/r^2$ . Corrected.
3. **Missing central term:** The pure two-body term was never added. Now included explicitly with the correct negative sign.

Final correct implementation:

```
a = (mu / r_sph**2) * dU_dr * r_hat
a += (mu / r_sph**2) * dU_dphi * phi_hat
a += (mu / r_sph**2) * (dU_dlam / max(cos_phi, 1e-12)) * lam_hat
a += -(mu / r_sph**2) * r_hat # central two-body term (inward)
return a
```

**Equation (general form):**

$$\mathbf{a} = -\nabla U, \quad U(r, \phi, \lambda) = \frac{\mu}{r} \left[ 1 + \sum_{n \geq 2} \left( \frac{a}{r} \right)^n \sum_{m=0}^n P_{nm}(\sin \phi) (C_{nm} \cos m\lambda + S_{nm} \sin m\lambda) \right]$$

$$a_r = -\frac{\partial U}{\partial r}, \quad a_\phi = -\frac{1}{r} \frac{\partial U}{\partial \phi}, \quad a_\lambda = -\frac{1}{r \cos \phi} \frac{\partial U}{\partial \lambda}$$

After these fixes, the model gives a surface gravity of 9.813 m/s<sup>2</sup> (correct central value with oblateness correction 0.07%), and a simulated ascent reaches a 315.6 × 424.5 km orbit, with final speed matching the theoretical circular speed to 0.1%.

**References:** Lemoine et al. [10]; Vallado [1, Ch. 8].

## 4.4 Aerodynamic Drag

**Code:** `forces.py` – `compute_drag_force`

```
mach = v_rel_norm / max(speed_of_sound, 1.0)
cd = np.interp(mach, C.MACH_BREAKPOINTS, C.CD_VALUES)
drag_magnitude = 0.5 * rho * cd * C.REFERENCE_AREA * v_rel_norm**2
return -drag_magnitude * v_rel_hat
```

**Equation:**

$$\mathbf{F}_{\text{drag}} = -\frac{1}{2} \rho C_D(M) A \|\mathbf{v}_{\text{rel}}\|^2 \hat{\mathbf{v}}_{\text{rel}}$$

**Explanation:** The drag vector opposes the relative velocity. The Mach-dependent drag coefficient table (peaking near  $M \approx 1.05$ ) models the transonic wave-drag hump, a well-known characteristic for launch-vehicle bodies. The values are reasonable engineering approximations.

**References:** Anderson [15, Ch. 4]; Braeunig [18, aerodynamics section].

## 4.5 Aerodynamic Lift

**Code:** `forces.py` – `compute_lift_force`

```
alpha = np.arctan2(v_transverse, abs(v_body[2]))
cl = compute_post_stall_normal_coefficient(alpha, cl_alpha, max_coefficient=1.2)
lift_mag = q_dyn * cl * C.REFERENCE_AREA
```

**Equation (lift magnitude):**

$$L = \bar{q} C_L A$$

**Explanation:** The lift force is computed from dynamic pressure, reference area, and a lift coefficient that depends on angle of attack (with post-stall modelling). This is the standard approach for aerodynamic force decomposition.

**References:** Anderson [15, Ch. 5]; Zipfel [6, Ch. 4].

## 4.6 Aerodynamic Moment (CP/CG Instability + Damping)

Code: `forces.py` – `compute_aerodynamic_moment`

```
Fn_mag = q_dyn * C.REFERENCE_AREA * cn
F_normal_body = -Fn_mag * u_trans
r_arm = np.array([0.0, 0.0, cp_pos_z - cg_pos_z])
torque_aero = np.cross(r_arm, F_normal_body)
damp_coeff = C.C_M_DAMPING * q_dyn * C.REFERENCE_AREA * C.REFERENCE_DIAMETER**2 / (2 * C.REFERENCE_DIAMETER)
torque_aero[0] += damp_coeff * omega[0]
torque_aero[1] += damp_coeff * omega[1]
```

**Equations:**

$$\mathbf{M}_{\text{static}} = \mathbf{r}_{\text{CP-CG}} \times \mathbf{F}_{\text{normal}}$$
$$M_{\text{damping},i} = C_{m_q} \bar{q} A d^2 \frac{\omega_i}{2V_{\text{rel}}} \quad (i = x, y)$$

**Explanation:** The static moment arises from the offset between centre of pressure and centre of mass – a classical static-margin instability (common in launch vehicles). The damping term uses a negative damping derivative  $C_{m_q}$  (here  $C_{m_q} = -8.0$ ) to produce a torque opposing angular velocity, matching standard rate-damping conventions.

**References:** Zipfel [6, Ch. 4]; Etkin & Reid [16, Ch. 5]; Tewari [7, Ch. 4].

## 4.7 Aerodynamic Heating (Sutton-Graves)

Code: `forces.py` – `compute_aerodynamic_heating`

```
return 1.7415e-4 * np.sqrt(rho / nose_radius) * v_rel_mag**3
```

**Equation:**

$$\dot{q}_s = k \sqrt{\frac{\rho}{R_n}} V^3, \quad k = 1.7415 \times 10^{-4} \text{ (SI units, Earth air)}$$

**Explanation:** The Sutton-Graves correlation is the standard engineering model for stagnation-point convective heat flux during atmospheric entry. The constant  $k$  is empirically determined for Earth air.

**References:** Sutton & Graves [13]; Anderson [15, Ch. 9].

## 4.8 Earth-Relative (Wind-Corrected) Velocity

Code: `utils.py` – `compute_relative_velocity`

```
omega_earth = np.array([0.0, 0.0, C.EARTH_ROTATION_RATE])
return v - np.cross(omega_earth, r) - wind
```

**Equation:**

$$\mathbf{v}_{\text{rel}} = \mathbf{v} - \boldsymbol{\omega}_{\oplus} \times \mathbf{r} - \mathbf{v}_{\text{wind}}$$

**Explanation:** This subtracts the velocity of the rotating atmosphere (and ground) at the vehicle’s position, giving the air-relative velocity used for aerodynamic forces. The cross-product order and sign are correct.

**References:** Vallado [1, Ch. 3]; Zipfel [6, Ch. 2].

## 5 Propulsion

### 5.1 Thrust with Altitude Compensation

Code: `forces.py` – `compute_thrust_force`

```
thrust_vac = ...
pressure_ratio = np.clip(P_amb / P0, 0.0, 1.2)
thrust_magnitude = thrust_vac - (thrust_vac - thrust_sl) * pressure_ratio
```

Equation (linear interpolation):

$$T = T_{\text{vac}} - (T_{\text{vac}} - T_{\text{sl}}) \frac{P_{\text{amb}}}{P_0}$$

which is equivalent to  $T = T_{\text{vac}} - (P_e - P_{\text{amb}})A_e$ .

**Explanation:** The nozzle back-pressure correction is accurately modelled by linearly interpolating between sea-level and vacuum thrust values as a function of ambient pressure. This reproduces the correct physical dependence.

**References:** Sutton & Biblarz [8, Ch. 3]; Humble, *Space Propulsion Analysis and Design*.

### 5.2 TVC Gimbal-Consistent Thrust Vector (Fix Applied)

Code: `forces.py` – `gimbaled_thrust_body_vector`

```
tvc_capacity = thrust_magnitude * np.sin(C.MAX_GIMBAL_ANGLE) * lever_arm
tvc_component_mag = min(torque_xy_mag, tvc_capacity)
tau_x, tau_y = torque_xy * (tvc_component_mag / torque_xy_mag)
f_y = tau_x / lever_arm; f_x = -tau_y / lever_arm
gimbal_angle = np.arcsin(np.hypot(f_x, f_y) / thrust_magnitude)
F_body = thrust_magnitude * [sin(gimbal_angle)*u_x, sin(gimbal_angle)*u_y, cos(gimbal_angle)*u_z]
```

Equations:

$$\boldsymbol{\tau} = \boldsymbol{\ell} \times \mathbf{F}_{\perp}, \quad \mathbf{F} = T(\sin \delta \hat{\mathbf{u}}_{\perp} + \cos \delta \hat{\mathbf{z}})$$

**Explanation:** Previously the thrust force always pointed along  $+Z$ , and the gimbal deflection only appeared as a torque. This fix makes the translational force consistent with the commanded torque: the lateral force components are derived from the torque via the lever arm, and the axial component is reduced by  $\cos \delta$ , conserving thrust magnitude. The gimbal angle is capped at the maximum physically allowed.

**References:** Sutton & Biblarz [8, Ch. 9]; Zipfel [6, Ch. 5].

### 5.3 Rocket Equation (Mass Flow Rate)

Code: `mass.py` – `compute_mass_flow_rate`

```
mdot = thrust / (isp * C.G0)
return -mdot * throttle
```

**Equation:**

$$I_{sp} = \frac{F}{\dot{m} g_0} \Rightarrow \dot{m} = -\frac{F}{I_{sp} g_0}$$

**Explanation:** The specific impulse definition is correctly rearranged. The negative sign indicates mass decrease during thrusting. A dry-mass floor correctly prevents further depletion once propellant is exhausted.

**References:** Sutton & Biblarz [8, Ch. 2]; Tsiolkovsky’s rocket equation.

## 6 Mass Properties

### 6.1 Centre of Mass (Station-Weighted Average)

**Code:** `mass.py` – `compute_center_of_mass`

```
masses = [s1_dry, s1_prop, s2_dry, s2_prop]
stations = [S1_DRY_CG_Z, S1_PROP_CG_Z, S2_DRY_CG_Z, S2_PROP_CG_Z]
return np.dot(masses, stations) / np.sum(masses)
```

**Equation:**

$$z_{cg} = \frac{\sum_i m_i z_i}{\sum_i m_i}$$

**Explanation:** The discrete-mass centre-of-mass formula is applied directly; it is recomputed at each time step as propellant mass changes.

**References:** Any standard statics/dynamics textbook (e.g., Beer & Johnston).

### 6.2 Inertia Tensor Interpolation (Fix Applied)

**Code:** `mass.py` – `compute_inertia_tensor` (stacked branch)

```
izz = C.IZZ_EMPTY + prop_frac * (C.IZZ_FULL - C.IZZ_EMPTY)
ixx = C.IXX_EMPTY + prop_frac * (C.IXX_FULL - C.IXX_EMPTY)    # was prop_frac**2
```

**Equation (linear interpolation):**

$$I_{xx} = I_{xx,empty} + f_p (I_{xx,full} - I_{xx,empty}), \quad I_{zz} = I_{zz,empty} + f_p (I_{zz,full} - I_{zz,empty})$$

where  $f_p$  is the propellant mass fraction.

**Explanation:** The “stacked” branch previously used a quadratic interpolation for  $I_{xx}$  while using linear for  $I_{zz}$ , which was inconsistent. The fix makes both linear, matching the other vehicle branches. The inequality  $I_{xx,full} \gg I_{zz,full}$  is preserved, correctly reflecting a slender rocket’s inertia.

**References:** No single public reference for interpolation; this is a standard engineering approximation used throughout the codebase.

## 7 Attitude Control

### 7.1 Quaternion PD/PID Control Law

**Code:** `control.py` – `pd_control_law`

```

tau_p = kp * q_error_vector
tau_d = -kd * omega
tau_i = ki * control_state.integral_error    # PID mode only
torque = tau_p + tau_d + tau_i

```

**Equation:**

$$\boldsymbol{\tau}_{\text{cmd}} = K_p \mathbf{q}_{e,v} - K_d \boldsymbol{\omega} + K_i \int \mathbf{q}_{e,v} dt$$

**Explanation:** Both proportional and derivative terms provide negative feedback (restoring and damping). The sign chain from rotation matrix convention through error quaternion to torque has been verified to be correct. Gains are scheduled proportionally to  $\min(I/I_{\text{ref}}, 1)$  to maintain a consistent natural frequency and damping ratio regardless of mass variation. Anti-windup is implemented via conditional integration (freeze integral on saturation) and a clamped integral budget.

**References:** Wie [5, Ch. 5]; Markley & Crassidis [3, Ch. 7]; Stevens & Lewis, *Aircraft Control and Simulation*.

## 8 Actuators

### 8.1 Rate-Limited Gimbal Slew (Rodrigues' Rotation Formula)

Code: `actuator.py` – `_limit_rotation`

```

return (cur_n * np.cos(theta) +
        np.cross(axis, cur_n) * np.sin(theta) +
        axis * np.dot(axis, cur_n) * (1 - np.cos(theta)))

```

**Equation (Rodrigues' formula):**

$$\mathbf{v}_{\text{rot}} = \mathbf{v} \cos \theta + (\hat{\mathbf{k}} \times \mathbf{v}) \sin \theta + \hat{\mathbf{k}} (\hat{\mathbf{k}} \cdot \mathbf{v}) (1 - \cos \theta)$$

**Explanation:** This rotates a unit vector by an angle  $\theta$  about axis  $\hat{\mathbf{k}}$ . The angle  $\theta$  is capped at  $\text{MAX\_GIMBAL\_RATE} \times dt$ , correctly modelling the finite-bandwidth servo. Units are radians throughout.

**References:** Vallado [1, Ch. 3]; any rigid-body kinematics text.

## 9 Reaction Control System (RCS)

### 9.1 RCS Torque and Propellant Mass Flow

Code: `rsc.py` – `compute_rcs_mass_flow_rate` and `rsc_available_torque`

```

max_torque = C.RCS_THRUST_PER_THRUSTER * num_thrusters * lever_arm_m
mdot_full = C.RCS_THRUST_PER_THRUSTER * num_thrusters / (isp_s * C.G0)
return mdot_full * min(rsc_torque_n_m / max_torque, 1.0)

```

**Equations:**

$$\tau_{\text{max}} = F_{\text{thr}} n \ell, \quad \dot{m} = \frac{F_{\text{thr}} n}{I_{sp} g_0} \cdot \frac{\tau_{\text{cmd}}}{\tau_{\text{max}}}$$

**Explanation:** The thruster couple torque is the product of thrust, number of thrusters, and lever arm. The mass flow is scaled linearly by the commanded torque fraction (duty cycle). An  $I_{sp}$  of 220 s is realistic for hydrazine/hypergolic thrusters. The sign convention here is positive consumption (subtracted from a propellant budget), intentionally different from the signed  $\dot{m}$  used in the ODE solver – this is explicitly documented to avoid confusion.

**References:** Wertz [4, Ch. 19]; Sutton & Biblarz [8, Ch. 10].

## 10 Navigation

### 10.1 Strapdown IMU Propagation

Code: `navigation.py` – `update_navigation_estimate`

```
g_accel = -C.MU_EARTH * r_current / r_norm**3
specific_force = true_accel_eci - g_accel
measured_specific_force = imu_propagator.measure_acceleration(specific_force, ...)
a_total_est = measured_specific_force + g_accel
propagated_velocity = previous_velocity + a_total_est * elapsed
propagated_position = previous_position + previous_velocity*elapsed + 0.5*a_total_est
```

Equations (strapdown INS convention):

$$\mathbf{f} = \mathbf{a} - \mathbf{g}, \quad \hat{\mathbf{a}} = \hat{\mathbf{f}} + \mathbf{g},$$

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \hat{\mathbf{a}}\Delta t, \quad \mathbf{r}_{k+1} = \mathbf{r}_k + \mathbf{v}_k\Delta t + \frac{1}{2}\hat{\mathbf{a}}\Delta t^2$$

**Explanation:** An accelerometer measures specific force (non-gravitational acceleration). Gravity is subtracted before the measurement model and added back after propagation – this is the correct INS mechanisation. The 2nd-order position update assumes constant acceleration over the step. Sensor noise is scaled as  $\sigma/\sqrt{\Delta t}$ , correctly discretising a continuous-time power spectral density.

**References:** Titterton & Weston [17]; Markley & Crassidis [3, Ch. 3].

## 11 Guidance

### 11.1 Gravity-Turn Pitch Program

Code: `guidance_common.py` – `gamma_profile_from_altitude`

```
h_norm = (altitude - turn_start_alt) / turn_scale
progress = np.tanh(h_norm)
gamma_deg = gamma_start + (gamma_final - gamma_start) * progress
```

Equation (smooth shaping):

$$\gamma(h) = \gamma_{\text{start}} + (\gamma_{\text{final}} - \gamma_{\text{start}}) \tanh\left(\frac{h - h_{\text{start}}}{h_{\text{scale}}}\right)$$

**Explanation:** This is an altitude-scheduled pitch program (not a re-derivation of the optimal gravity-turn differential equation). It provides a smooth  $C^1$  transition from

a vertical launch to a prescribed pitch angle at main engine cutoff, which is a standard practical method.

**References:** Braeunig [18, “Gravity Turn” section]; Tewari [7, Ch. 7].

## 11.2 Vis-Viva Circularisation

**Code:** `_guidance_orbit.py` – orbit insertion CIRCULARIZE mode

```
vis_viva_term = 2.0 / r_mag - 1.0 / desired_radius
target_tangential_speed = np.sqrt(C.MU_EARTH * vis_viva_term)
dv_to_target_orbit = target_tangential_speed - v_horiz
```

**Equation (vis-viva):**

$$v^2 = \mu \left( \frac{2}{r} - \frac{1}{a} \right)$$

**Explanation:** For a circular target orbit, the semi-major axis  $a$  equals the desired radius. The equation gives the required tangential speed at the current radius. The velocity deficit drives the throttle command (positive deficit  $\rightarrow$  burn prograde).

**References:** Vallado [1, Ch. 2]; Curtis [2, Ch. 2].

## 11.3 Effective-Gravity Apogee-Raise Pitch Solve

**Code:** `_guidance_orbit.py` – orbit insertion RAISE\_APOGEE mode

```
g_eff = (C.MU_EARTH / r_mag**2) - (v_horiz**2 / r_mag)
sin_pitch = (a_req_vert + g_eff) / a_thrust
```

**Equation:**

$$g_{\text{eff}} = \frac{\mu}{r^2} - \frac{v_{\perp}^2}{r}$$

**Explanation:** This is the net inward (centripetal-corrected) gravitational acceleration along the local vertical. The radial equation of motion in polar coordinates is  $\ddot{r} - r\dot{\theta}^2 = -\mu/r^2$ , so rearranging gives exactly this expression. The pitch angle is then solved to achieve a required vertical acceleration.

**References:** Curtis [2, Ch. 2]; Vallado [1, Ch. 3].

## 11.4 Suicide-Burn Ignition Altitude (Variable-Mass)

**Code:** `recovery.py` – `estimate_suicide_burn`

```
g_local = C.MU_EARTH / r_norm**2
thrust_accel = thrust_newton / mass_kg
a_brake = thrust_accel - g_local
# dv_ideal = Isp*g0*ln(m0/mf)
# a_mean = F*ln(m0/mf)/(m0-mf) - g
# h_ignite = v_eff**2 / (2*a_mean)
```

**Equation (constant-deceleration stopping distance, refined):**

$$h_{\text{ignite}} = \frac{v_{\text{eff}}^2}{2a_{\text{mean}}}, \quad a_{\text{mean}} = \frac{F \ln(m_0/m_f)}{m_0 - m_f} - g$$

where  $v_{\text{eff}}$  is the current velocity, and  $a_{\text{mean}}$  is the average deceleration over a burn that consumes propellant, obtained from the Tsiolkovsky equation.

**Explanation:** This improves upon the naive  $v^2/(2a)$  by accounting for the mass decrease during the burn, yielding a more accurate altitude at which to initiate the final landing burn. The deceleration is the thrust acceleration minus gravity.

**References:** Klumpp [14]; Braeunig [18, powered descent section]; Apollo Lunar Descent Guidance.

## 11.5 Zero-Effort-Miss / Zero-Effort-Velocity (ZEM/ZEV) Divert Law

**Code:** `guidance_booster.py` – terminal divert guidance

```
zem = impact_miss
a_divert = (6.0 / t_go_capture**2) * zem + (2.0 / t_go_capture) * v_horiz_site
```

**Equation:**

$$\mathbf{a}_{\text{cmd}} = \frac{6}{t_{go}^2} \mathbf{ZEM} + \frac{2}{t_{go}} \mathbf{ZEV}$$

**Explanation:** This is the classical ZEM/ZEV feedback guidance law, which drives both predicted miss distance and terminal velocity error to zero simultaneously under a constant-acceleration assumption over the remaining time-to-go. The coefficients  $\frac{6}{t_{go}^2}$  and  $\frac{2}{t_{go}}$  are standard. Project-specific acceleration caps and biases are added on top; the implementation honestly declares itself as ZEM/ZEV-inspired.

**References:** Klumpp [14]; Apollo guidance literature; Lu, *Propellant-Optimal Powered Descent Guidance*.

## 12 Physics Sanity Checks

### 12.1 Total Mechanical Energy (Two-Body Form)

**Code:** `physics_checks.py` – `compute_total_energy`

```
kinetic = 0.5 * m * v_norm**2
potential = -C.MU_EARTH * m / r_norm
return kinetic + potential
```

**Equation:**

$$E = \frac{1}{2}mv^2 - \frac{\mu m}{r}$$

**Explanation:** This is the correct two-body energy expression, valid from the surface to orbit. It is not the linearised  $mgh$  form, which is only valid for small altitudes. The energy is monitored during ballistic coast phases to detect integrator drift.

**References:** Vallado [1, Ch. 1]; Curtis [2, Ch. 2].

## 12.2 Quaternion Norm Check

**Code:** `physics_checks.py – check_quaternion_norm`

```
norm = np.linalg.norm(q)
if abs(norm - 1.0) > tolerance:
    raise ValidationError(...)
```

**Equation:**

$$\|\mathbf{q}\| = 1 \quad (\text{unit quaternion constraint})$$

**Explanation:** A quaternion representing a pure rotation must have unit norm. Although RK4 does not preserve this exactly, the check combined with explicit re-normalisation after every sub-stage (see Section 3.1) ensures the constraint is satisfied to within tolerance.

**References:** Markley & Crassidis [3, Ch. 2].

## References

1. D.A. Vallado, *Fundamentals of Astrodynamics and Applications*, 4th ed., Microcosm Press / Springer, 2013.
2. H.D. Curtis, *Orbital Mechanics for Engineering Students*, 4th ed., Butterworth-Heinemann, 2020.
3. F.L. Markley and J.L. Crassidis, *Fundamentals of Spacecraft Attitude Determination and Control*, Springer, 2014.
4. J.R. Wertz (ed.), *Spacecraft Attitude Determination and Control*, D. Reidel / Kluwer, 1978.
5. B. Wie, *Space Vehicle Dynamics and Control*, 2nd ed., AIAA Education Series, 2008.
6. P.H. Zipfel, *Modeling and Simulation of Aerospace Vehicle Dynamics*, 3rd ed., AIAA Education Series, 2014.
7. A. Tewari, *Atmospheric and Space Flight Dynamics*, Birkhäuser, 2007.
8. G.P. Sutton and O. Biblarz, *Rocket Propulsion Elements*, 9th ed., Wiley, 2016.
9. NOAA, NASA, USAF, *U.S. Standard Atmosphere, 1976*, NOAA-S/T 76-1562, U.S. Government Printing Office, 1976.
10. F.G. Lemoine et al., *The Development of the Joint NASA GSFC and NIMA Geopotential Model EGM96*, NASA/TP-1998-206861, 1998.
11. S.W. Shepperd, “Quaternion from Rotation Matrix,” *Journal of Guidance and Control*, Vol. 1, No. 3, 1978, pp. 223-224.
12. W.H. Press, S.A. Teukolsky, W.T. Vetterling, B.P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed., Cambridge University Press, 2007.

13. K. Sutton and R.A. Graves, *A General Stagnation-Point Convective-Heating Equation for Arbitrary Gas Mixtures*, NASA TR R-376, 1971.
14. A.R. Klumpp, “Apollo Lunar Descent Guidance,” *Automatica*, Vol. 10, 1974, pp. 133-146.
15. J.D. Anderson, *Fundamentals of Aerodynamics*, 6th ed., McGraw-Hill, 2017.
16. B. Etkin and L.D. Reid, *Dynamics of Flight: Stability and Control*, 3rd ed., Wiley, 1996.
17. D.H. Titterton and J.L. Weston, *Strapdown Inertial Navigation Technology*, 2nd ed., IEE Radar, Sonar and Navigation Series, 2004.
18. J.P. Braeunig, *Rocket and Space Technology* (braeunig.us), aerodynamics and gravity-turn sections, accessed 2026.